A Component-Wise Intuition for the FFT Trick in State-Space Models

I was struggling to understand how the FFT trick works for State-Space Models (SSMs), especially the idea of viewing the RNN outputs at each time step as sequences evolving component-by-component (this is from lecture 17). I rewatched the lectures but still wasn't fully clear on what this perspective buys us. To work through the confusion, I drew a diagram and had an extended interaction with ChatGPT, which helped me connect the intuition: each coordinate of the hidden state forms its own time-series, and the FFT lets us convolve these sequences efficiently. I'm sharing my annotated conversation and the diagram here in case it helps anyone else build intuition for how the component-wise viewpoint makes the FFT trick work.

This is the trace without any annotations (sorry about any spelling errors!):
https://chatgpt.com/share/6938b61a-b838-800f-bd21-b9b7a0b6807b

inputed a screenshot of the notes I took during lecture. These are going over how we can think of y_t (the output at each time step), in terms of components in a time-evolution.

Here y_t has d_y dimension and x_t has d_x dimension.

I am asking about what does it mean to have a sequence. y_t is described as a stack of sequence change in time, but I was unclear what this means

What does this mean...now sure what its saying about the sed

Here's a **clean, structured explanation** of what your handwritten notes are saying — especially the part about the **"seq of scalars vs seq of vectors", convolution, and why this lets you go from** $T^2 \to T \log T$.

I'll rewrite the ideas in your notes in a much clearer conceptual form.

⭐ 1. State h_t stores ALL past information — but as a vector

Your notes write:

- $h_t \in \mathbb{R}^{d_h}$ Correctly wrote that h_t has n dimensions
- each component of h_t is **one scalar sequence evolving over time**

This is the key idea:

👉 Even though h_t is a vector, you can think of it as

$$h_t = \left(h_t^{(1)}, h_t^{(2)}, \ldots, h_t^{(d_h)}\right)$$

and each coordinate (superscript) evolves through time:

$$\{h_0^{(i)},\ h_1^{(i)},\ \ldots,\ h_T^{(i)}\}.$$

So:

- You have **d_h independent scalar sequences**, one per dimension.
- Each is updated by the SSM recurrence.

This is what your note calls:

> **"streak of sequences of scalars"**

⭐ 2. Output y_t also has d_y components → so also d_y scalar sequences

Your output:

$$y_t \in \mathbb{R}^{d_y}$$

has components:

$$(y_t^{(1)}, y_t^{(2)}, \ldots, y_t^{(d_y)})$$

so each component is its own scalar time-series:

$$y_0^{(i)}, y_1^{(i)}, y_2^{(i)}, \ldots$$

Your notes write:

> "Think of time-evolution on a component-by-component basis."
> "This is a sequence."

Exactly — every output dimension is just **one scalar sequence evolving over time.**

⭐ 3. Each scalar output sequence is a convolution over scalar input sequences

Your notes highlight this formula:

$$y_t^{(i)} = \sum_{k=0}^{t} F_k \, x_{t-k}$$

This means:

- Fix one output coordinate i
- It is **just** a convolution between:
  - a kernel sequence $\{F_0, F_1, \ldots, F_t\}$
  - a scalar input sequence $\{x_0, x_1, \ldots, x_t\}$

Your notes color this:

> "Sequence (y[i]) = ∑ F_k * Sequence(x[i])
> convolution filter"

Got to the core idea behind this trick. Since we view each component in the vector as a sequence in time, each of these components are independent of each other and can be used in the FTT. At this point, it gave a good high level understanding, but I was still confused.

The idea:

Every output component evolves independently as a convolution of the corresponding input components.

This is the critical conceptual shift.

⭐ 4. Why the notes say: "Instead of seq of vectors... want seq of scalars"

Right now, the SSM state equation works with **vectors**, e.g.:

$$h_{t+1} = Ah_t + Bx_t.$$

This is hard to compute for long sequences because:

- the matrix power A^k appears,
- computations become sequential.

But if you can transform the system so that:

The state update decouples into independent scalar recurrences

(i.e., A is diagonal or diagonalizable)

then each dimension becomes:

$$h_{t+1}^{(i)} = \lambda_i h_t^{(i)} + b_i x_t$$

a **1D recurrence**.

Properly explains why looking at it in a component basis perspective allows us to decouple the recurrences. I am glad that it first started with the traditional SSM recurrence relationship and then explained how the decoupled relationship would look like.

Your note calls this:

> "instead of seq of vectors... want seq of scalars so can do conv on them"

Meaning:

- Vector recurrence = expensive matrix multiplication
- Scalar recurrence = cheap and parallel

When turned into scalar sequences, each sequence can be convolved using FFT.

⭐ 5. That's where the $T^2 \to T \log T$ comes from

If you treat the output as **vector sequences**, convolution costs:

$$O(d_y T^2)$$

But if each dimension is treated as a **scalar sequence**, convolution via FFT costs:

$$O(d_y T \log T)$$

your notes conclude:

> "so can go from $T^2 \to T \log T$".

Because:

$$\mathbf{FFT(convolution)} = O(T \log T)$$

Just states this as a fact rather than explaining the intuition -- this is fine for now

🎯 Final Summary

Your notes are saying:

1. y_t and h_t are vectors, but each coordinate of them is **just a scalar time-series**.
2. The SSM recurrence implies each output scalar sequence is a **convolution filter applied to the input scalar sequences.**
3. If A becomes **diagonal or diagonalizable**, the recurrence breaks into **independent 1D recurrences**.
4. 1D recurrences = 1D convolutions.
5. 1D convolutions can be computed with FFT → **fast**.
6. So operating on **scalar sequences instead of vector sequences** is crucial to achieving $T^2 \to T \log T$.

If you want, I can redraw your entire notes as a **clean digital diagram**, or walk through each colored region line-by-line.

While chatGPT gave a good explanation, I was still confused about what the notation meant. Where we looking at different layers? just one layer but different times?

so t is across the time...

like if we are looking for h_5 = all the info in H-4, h3,h2,h1....is that the idea? confused with the superscript and underscript

Yes — **t is the time index**, and all the superscripts/underscripts in your notes are just referring to:

1. **which time step** you are at (the subscript: h_t)
2. **which coordinate** of the vector you are talking about (the superscript: $h_t^{(i)}$)

Let me break this down cleanly so you see exactly what each symbol means and how "memory" works.

✅ 1. What does the subscript mean? (time index)

$$h_t$$

The **subscript** t always means:

"the state AFTER seeing inputs up to time t"

i.e.,

$$h_5 = \text{state after seeing } x_1, x_2, x_3, x_4, x_5.$$

This state h_5 contains information aggregated from:

- h_4 (which contains info from h_3)
- which contains info from h_2
- etc.

So yes, your intuition is right:

h_5 contains everything that flowed through $h_4, h_3, h_2, h_1, \ldots$

because the recurrence:

$$h_{t+1} = Ah_t + Bx_t$$

propagates history forward.

Explicitly explained that ht just contains information from all the previous time steps. Aligns with the note that the hidden state contains the "memory" of the network

✅ 2. What does the superscript mean? (vector coordinate)

A state h_t is a **vector** in $\mathbb{R}^{d_h}$.

Write it explicitly:

$$h_t = \begin{bmatrix} h_t^{(1)} \\ h_t^{(2)} \\ \vdots \\ h_t^{(d_h)} \end{bmatrix}$$

Here:

- **superscript = which component of the vector**
- **subscript = which time step**

So:

Symbol	Means
h_5	the FULL state vector after step 5
$h_5^{(3)}$	the **3rd element** of the state vector at time 5
$h_t^{(i)}$	the **i-th scalar sequence** evolving across time

This is what your notes call:

> "Think of time evolution on a component-by-component basis."

✅ 3. Why this matters: turning the vector recurrence into many scalar recurrences

The SSM recurrence:

$$h_{t+1} = Ah_t + Bx_t$$

acts on vectors.

But each coordinate behaves like a **scalar time series**:

$$h_0^{(i)},\ h_1^{(i)},\ h_2^{(i)}, \dots$$

Your notes emphasize this:

> "Streak of sequences of scalars"

because if A is diagonal(izable), then:

$$h_{t+1}^{(i)} = \lambda_i\, h_t^{(i)} + b_i x_t.$$

This means:

- each coordinate evolves **independently,**
- and the entire SSM becomes a stack of many 1D recurrences.

That's why the superscripts matter.

Connected it back to why this perceptive matter and why we are looking at it in a coordinate basis -- want each coordinate to evolve independently with time

⭐ 4. The key picture (this will clear up everything)

FULL VECTOR STATE over time:

$$h_0 = (h_0^{(1)}, h_0^{(2)}, \dots, h_0^{(d_h)})$$
$$h_1 = (h_1^{(1)}, h_1^{(2)}, \dots, h_1^{(d_h)})$$
$$h_2 = (h_2^{(1)}, h_2^{(2)}, \dots, h_2^{(d_h)})$$
$$\vdots$$

If you follow one coordinate, you get a scalar sequence:

$$h_0^{(i)},\ h_1^{(i)},\ h_2^{(i)},\ \dots$$

This is what becomes a **convolution** and where FFT applies.

🎯 Final Intuition (1 sentence)

- **Subscript** t = time
- **Superscript** i = coordinate of the hidden state vector
- h_t contains *all information* from all earlier states due to recurrence
- Each coordinate $h_t^{(i)}$ is its own scalar time-evolving sequence
  → this is why we can diagonalize and use FFT.

If you want, I can draw a diagram that shows:

- the state flowing across time,
- how each coordinate evolves,
- and how this becomes convolution.

in the y are they teh layers and seq?

Ah—**this is the source of the confusion.**
When your notes write things like:

$$y_t = \begin{bmatrix} y_t^{(1)} \\ y_t^{(2)} \\ \dots \\ y_t^{(d_y)} \end{bmatrix}$$

you are wondering:

> "Are these **layers** or **sequences?**"

They are *NOT* layers.
They are *NOT* separate time steps.
They are the *components* (dimensions) of the output vector at time t.

Let me make this crystal clear.

✅ What the y superscript means

At each time step t, the SSM produces a **d_y-dimensional output vector**:

$$y_t \in \mathbb{R}^{d_y}.$$

This means:

- $y_t^{(1)}$ = the 1st coordinate of the output at time t
- $y_t^{(2)}$ = the 2nd coordinate
- ...
- $y_t^{(d_y)}$ = the last coordinate

So superscripts = output dimensions, not layers.

If $d_y = 512$, then each y_t is a 512-dimensional vector.

✅ What the sequence is

If you zoom into one dimension of the output (fix i):

$$y^{(i)} = (y_0^{(i)}, y_1^{(i)}, y_2^{(i)}, \dots)$$

This *across time* is what your notes call a:

> "sequence for component i"

So:

Along time → sequence
Across coordinates → vector dimension

🧠 So what are the *layers* then?

Layers are the matrices:

- A: state update
- B: input projection
- C: output projection
- D: skip/linear term

These are **not** indexed by superscripts.
They are fixed matrices, not sequences.

✓ Putting it all together visually

At each time t:

```cpp
y_t = [ y_t^(1), y_t^(2), ..., y_t^(d_y) ]ᵀ
```

These are **components of the output vector**, NOT layers.

Across time:

For each coordinate i,

```css
y_0^(i), y_1^(i), y_2^(i), ...
```

is a **sequence**, and **each sequence is computed by convolution**.

That's why your notes say:

> "Sequence (y[i]) = ∑ F_k * Sequence(x[i])"

They are talking about **scalar sequences** per output dimension.

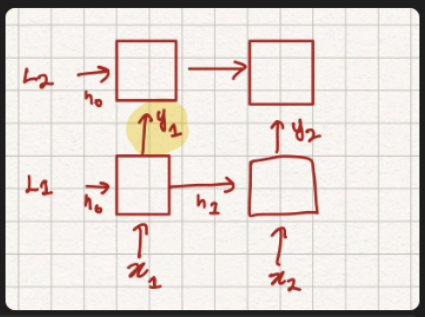
⭐ Final Answer

- **The y superscripts do *not* represent layers.**
- **They represent the individual coordinates of the output vector.**
- Each coordinate is a **scalar sequence over time**, not a neural layer.
- These scalar sequences are what get convolved with the SSM kernel (and where FFT is used).

If you want, I can draw a complete visual diagram showing:

- how h_t expands over time
- how each coordinate becomes its own scalar sequence
- how the convolution arises
- and where diagonalizing A helps.

I am a visual learner and I was struggling to understand where the y_t fit in this basic RNN block diagram. I highlighted y1 (the output of the first RNN block at timestep1) and wanted to double check if my understanding was correct



so the y is one of thoes outputs? like one output at one time step

Yes — **exactly.**
In your diagram, each **y_1, y_2, ...** is the **output at a single time step**.

Let me map your drawing directly to the SSM equations so it clicks perfectly.

✅ What the diagram shows

At time step t = 1:

- Input: x_1
- Hidden state computed: h_1
- Output: y_1

At time step t = 2:

- Input: x_2
- Hidden state computed: h_2
- Output: y_2

And so on.

So the arrow pointing up toward y_1 (the yellow highlighted one) is:

$$y_1 = Ch_1 + Dx_1.$$

And the arrow pointing up to y_2 is:

$$y_2 = Ch_2 + Dx_2.$$

🎯 So the answer is:

✔ **Yes, each y_t is one output vector at time step t.**

✔ **Each box emits one y_t.**

✔ **That is NOT a layer—it's the output at that time step.**

📌 What the hidden state boxes represent

Each hidden state update:

$$h_t = Ah_{t-1} + Bx_t$$

is the *same* recurrence applied at every time step.

So the boxes you've drawn vertically are **time steps**, NOT layers.

Side note: one thing I noticed is that it likes to ramble about everything related to the topic/question. For instance, my question was simple -- is the y1 I draw in the diagram referring to the y_t we have in our formula. While ChatGPT answered it, it continued talking about hidden states again. While learning, this can be time consuming as you are trying to piece together a bunch of topics at once. Maybe in study mode ChatGPT might give more succinct answers.

📌 Everything in one picture

Time step 1:

```
x1 → [ state update box ] → h1
                  ↑
                  y1 (output at time 1)
```

Time step 2:

```
x2 → [ state update box ] → h2
                  ↑
                  y2 (output at time 2)
```

Started responding with more diagrams when I inputed two pictures before. This was helpful

---

🔍 Important clarification about y superscripts from before

Earlier we talked about $y_t^{(i)}$.

That is **NOT** another time step.

- subscript t = time
- superscript i = which coordinate of the output vector

For example, if $y_1 \in \mathbb{R}^{512}$, then:

$$y_1 = \begin{bmatrix} y_1^{(1)} \\ y_1^{(2)} \\ \dots \\ y_1^{(512)} \end{bmatrix}$$

These are the **512 numbers** inside the output vector y_1.

⭐ Final Answer

✔ **Yes — the y in your diagram is the output at a single time step.**

✔ **Each time you feed in x_t, you get a new h_t and a new y_t.**

✔ **The superscripts on y refer to components of the output vector, not additional time steps or layers.**

If you want, I can redraw your diagram in a clean formal SSM diagram with labels for:

- x_t
- h_t
- y_t
- A, B, C, D matrices
- and how convolution / FFT enters when unrolling over time.

y[K] = y1[k] y2[k] ...yt[k]

tink of itme evoltuion/sequence on a component by compoenet basis...what does this mean??

Great question — **this is the key conceptual jump** in SSMs, and once you get this, everything about diagonalization + FFT makes sense.

Let's decode the exact line from your notes:

> **"y[k] = y¹[k], y²[k], ..., y^{d_y}[k].**
> **Think of time evolution / sequence on a component-by-component basis."**

I got what component by component basis meant, but wasn't sure why we are combining different time steps into one

You're asking:

> What does "component-by-component sequence" mean?

Here is the clean explanation.

🌟 1. At each time step k, the output y[k] is a vector

If the model outputs a vector of dimension d_y, then:

$$y[k] = \begin{bmatrix} y^{(1)}[k] \\ y^{(2)}[k] \\ \vdots \\ y^{(d_y)}[k] \end{bmatrix}$$

This is *one* output at time **k**, but it has d_y components.

So far, obvious.

🌟 2. Now imagine zooming in on just one component (say the first one)

Instead of looking at the full vector, we track **how one coordinate changes over time**:

$$\underbrace{y^{(1)}[0],\ y^{(1)}[1],\ y^{(1)}[2],\ \dots}_{\text{A scalar sequence}}$$

This is a **time series** — one scalar value per time step.

This is what your notes call:

> **the time evolution of component 1**

🌟 3. Do this for every component of y

For component 2:

$$y^{(2)}[0],\ y^{(2)}[1],\ y^{(2)}[2],\ \dots$$

This was super helpful to see how we are just lining up the same components across timesteps

For component 3:

$$y^{(3)}[0],\ y^{(3)}[1],\ y^{(3)}[2],\ \dots$$

...
until component d_y:

$$y^{(d_y)}[0], y^{(d_y)}[1], \dots$$

🌟 4. BIG IDEA:

Each coordinate of the output is its own scalar sequence evolving over time.

So instead of seeing **one vector sequence**:

$$y[0], y[1], y[2], \ldots$$

you can view it as **d_y separate scalar sequences**:

$$\begin{aligned} \text{Sequence 1:} \quad & y^{(1)}[0], y^{(1)}[1], y^{(1)}[2], \ldots \\ \text{Sequence 2:} \quad & y^{(2)}[0], y^{(2)}[1], y^{(2)}[2], \ldots \\ & \vdots \\ \text{Sequence } d_y: \quad & y^{(d_y)}[0], \ldots \end{aligned}$$

This is what your notes mean by:

> **think of time evolution on a component-by-component basis**

🌟 5. Why do this?

Because the SSM equation shows **each component obeys a convolution**:

$$y^{(i)}[k] = \sum_{\tau=0}^{k} F_{\tau}^{(i)} \cdot x[k-\tau]$$

This is a one-dimensional convolution per component.

👉 And **1D convolution** is what FFT can accelerate from

$$O(T^2) \rightarrow O(T \log T).$$

This only works if you think of each output component as its own **scalar time-series**.

🎯 FINAL INTUITION SUMMARY

When you write y[k], that's the vector at time k.

When you write y^1[k], y^2[k], ... those are the components of the vector.

If you track each one across time, you get d_y separate scalar sequences.

Each of these scalar sequences is computed by a convolution.

Convolutions can be computed fast using FFT → this gives the speedup.

If you want, I can draw a picture that shows:

- How the vector sequence splits into scalar sequences
- How each scalar sequence is a convolution
- How FFT operates on each sequence

Just say "diagram please."

Generate a diagram of how this sequence works... so the y1 I highlighted... that's the output right and that is d_y dim ....now what

Below is a **clear diagram** showing EXACTLY what is happening in your notes — including where your highlighted $\mathbf{y_1}$ lives, how it becomes **d_y scalar sequences**, and how this leads to FFT-based convolution.

I'll give you *two diagrams*:

1. **Time-step view** (what your drawing is showing)
2. **Component-by-component view** (what the sentence "think of time evolution on a component-by-component basis" actually means)

✅ 1. TIME-STEP VIEW (your drawing)

At each time step, the SSM outputs **one vector** $y_t \in \mathbb{R}^{d_y}$.

```cpp
t = 1                          t = 2
──────────────────        ──────────────────
x₁ → [ State Update ] → h₁ → [ Next State ] → h₂
            |                          |
            ▼                          ▼
           y₁                         y₂
     (a d_y-dimensional vector)  (a d_y-dimensional vector)
```

So your highlighted $\mathbf{y_1}$ is:

$$y_1 = \begin{bmatrix} y^{(1)}[1] \\ y^{(2)}[1] \\ \vdots \\ y^{(d_y)}[1] \end{bmatrix}$$

Just a vector at **time = 1**.

So far so good.

🧠 But the notes say:

> "Think of time evolution / sequence on a component-by-component basis"

This refers to a **different view**:

✅ 2. COMPONENT-BY-COMPONENT VIEW (the key idea)

Take one coordinate of each output vector and line them up across **time**.

Component 1 (i = 1):

```
scss
y¹[1] → y¹[2] → y¹[3] → …   (scalar sequence)
```

Component 2 (i = 2):

```
scss
y²[1] → y²[2] → y²[3] → …   (scalar sequence)
```

…

Component d_y:

```
scss
yᵈʸ[1] → yᵈʸ[2] → yᵈʸ[3] → …   (scalar sequence)
```

Here is the **diagram combining BOTH views:**

```
graphql
```

```
            TIME AXIS →
         t=1        t=2        t=3        t=4
       ———————    ———————    ———————    ———————
y¹:    y¹[1]  →   y¹[2]  →   y¹[3]  →   y¹[4]   (scalar sequence)
y²:    y²[1]  →   y²[2]  →   y²[3]  →   y²[4]   (scalar sequence)
y³:    y³[1]  →   y³[2]  →   y³[3]  →   y³[4]   (scalar sequence)
 ...     ...        ...        ...        ...
yᵈʸ: yᵈʸ[1] → yᵈʸ[2] → yᵈʸ[3] → yᵈʸ[4]       (scalar sequence)
```

This was a super helpful diagram. It helped me put together how all the components are lined up across time, and helped me redraw/modify my notes

🚨 IMPORTANT:

Each **row** is a **separate scalar time-series**.

Each row follows its own convolution:

$$y^{(i)}[t] = \sum_{k=0}^{t} F_k^{(i)}\, x[t-k]$$

And THIS form is what lets us apply **FFT**.

🧩 Putting the two diagrams together

At each time step:

```
y₁ = [ y¹[1], y²[1], …, y^{d_y}[1] ]ᵀ
y₂ = [ y¹[2], y²[2], …, y^{d_y}[2] ]ᵀ
```

Across time:

Each coordinate becomes its own **independent scalar sequence**:

```
(y¹[1], y¹[2], y¹[3], …)
(y²[1], y²[2], y²[3], …)
...
```

This is exactly what "sequence component-by-component" means.

🎯 FINAL SUMMARY

✔️ **Your highlighted y_1 is the full output vector at time step 1.**

✔️ **That vector contains d_y scalar components.**

✔️ **If you track each component across time → you get d_y independent sequences.**

✔️ **Each sequence is a convolution between the input and the SSM kernel.**

✔️ **Convolution ⇒ FFT ⇒ $T^2 \to T \log T$.**

If you'd like, I can generate:

- A **prettier PDF-style diagram**
- A **tensor diagram** (like in ML papers)
- A **diagram showing diagonalization of A → independent sequences**

Just tell me!

so now we apply a similar idea all the past time steps of x; xt has a. dx dim. explain how this and the diagram you drew lead us to the equation seq(y[k]) = sum l=1 --> dx F_kl * seq(x[l])

Here I am asking about how we apply this same idea to look at the vectors in a component basis for the inputs x_t

Perfect — this is **exactly the step** that ties your notes together:

We already understood that **each component of y across time** is its own scalar sequence:

$$y^{(i)}[0],\ y^{(i)}[1],\ y^{(i)}[2],\ \dots$$

Now we must understand:

❓ **How do we get this equation from your notes?**

$$\mathrm{Seq}(y[k]) \;=\; \sum_{\ell=1}^{d_x} F_{k\ell}\ \mathrm{Seq}(x[\ell])$$

And what it is saying.

🚀 Step 1 — Remember: x_t also has components

Just like y_t has dimension d_y, the input has dimension:

$$x_t \in \mathbb{R}^{d_x}.$$

So write its components:

$$x_t = \begin{bmatrix} x^{(1)}[t] \\ x^{(2)}[t] \\ \vdots \\ x^{(d_x)}[t] \end{bmatrix}$$

🔥 **Each component $x^{(\ell)}[t]$ is ALSO a scalar sequence over time:**

$$x^{(\ell)}[0],\ x^{(\ell)}[1],\ x^{(\ell)}[2],\ \ldots$$

Just like we did for the output!

So we have **d_x independent scalar sequences** of input evolving across time.

🚀 Step 2 — The SSM update mixes all input components into the state

State update:

$$h_{t+1} = Ah_t + Bx_t$$

where B is a matrix of shape:

$$B \in \mathbb{R}^{d_h \times d_x}.$$

This means each component of the input contributes to the new state:

$$h_{t+1}^{(i)} = \sum_{\ell=1}^{d_x} B_{i\ell}\, x^{(\ell)}[t] \ + \ (Ah_t)^{(i)}$$

So **each coordinate** of the input affects **each coordinate** of the state through B.

🚀 Step 3 — The output y_t is a linear readout of the state

$$y_t = Ch_t + Dx_t$$

Expand one component:

$$y^{(k)}[t] = \sum_{i=1}^{d_h} C_{ki}\, h_t^{(i)} \ + \ \sum_{\ell=1}^{d_x} D_{k\ell}\, x^{(\ell)}[t].$$

Thus:

Each output component is a linear combination of

all components of the state and the input.

Nice explanation for how both all the past inputs and the past hidden states affect the current state

---

🚀 Step 4 — Unroll the recurrence → convolution

If you unroll the SSM over time:

$$h_t = A^{t-1}Bx_1 + A^{t-2}Bx_2 + \cdots + A^0 Bx_t$$

Then:

$$y_t = Ch_t + Dx_t$$

becomes:

$$y_t = \sum_{\ell=1}^{d_x} \left(\sum_{k=0}^{t} CA^k B_{:,\ell}\, x^{(\ell)}[t-k] \right) + \sum_{\ell=1}^{d_x} D_{:,\ell}\, x^{(\ell)}[t].$$

Group by input component ℓ:

$$y_t^{(i)} = \sum_{\ell=1}^{d_x} \left(\sum_{k=0}^{t} F_{k,\ell}^{(i)}\, x^{(\ell)}[t-k] \right)$$

where

$$F_{k,\ell}^{(i)} = C_{i,:} A^k B_{:,\ell}$$

are the convolution **filters**.

🎉 Step 5 — Finally we get the equation in your notes

If you collect the entire **sequence** of the i-th output component across time:

$$\mathrm{Seq}(y^{(i)}) = \left(y^{(i)}[0], y^{(i)}[1], \dots\right)$$

then:

$$\boxed{\mathrm{Seq}(y^{(i)}) = \sum_{\ell=1}^{d_x} F_{:,\ell}^{(i)} \;*\; \mathrm{Seq}(x^{(\ell)})}$$

In words:

✓ **Each output component is a sum over**

✓ **convolutions between its corresponding filter $F_{:,\ell}$**

✓ **and each input component's sequence.**

This is exactly the line in your notes:

$$\mathrm{Seq}(y[k]) = \sum_{\ell=1}^{d_x} F_{k\ell}\, \mathrm{Seq}(x[\ell]).$$

📌 Visual DIAGRAM tying it together

Here's the **big picture**:

perl Copy code

```
        INPUT SEQUENCES (dₓ of them)
    x¹[1] → x¹[2] → x¹[3] → …
    x²[1] → x²[2] → x²[3] → …
     ...
    xᵈˣ[1] → xᵈˣ[2] → xᵈˣ[3] → …

              | (each passes through its
              |  own convolution filter Fᵢ)
              v

    For output component i:
    y⁽ⁱ⁾ =   F₁⁽ⁱ⁾ * Seq(x¹)
          + F₂⁽ⁱ⁾ * Seq(x²)
          + ···
          + F_{dₓ}⁽ⁱ⁾ * Seq(xᵈˣ)

    This gives:
    y⁽ⁱ⁾[1] → y⁽ⁱ⁾[2] → y⁽ⁱ⁾[3] → …
```

Really nice diagram that helped me solidify my understanding about how the inputs and convolution filters are impacting each component in an output.

So each output component is:

- **sum over all input components**
- **each passed through its own convolution kernel**
- **producing d_y scalar sequences**

🎯 FINAL INTUITION SUMMARY

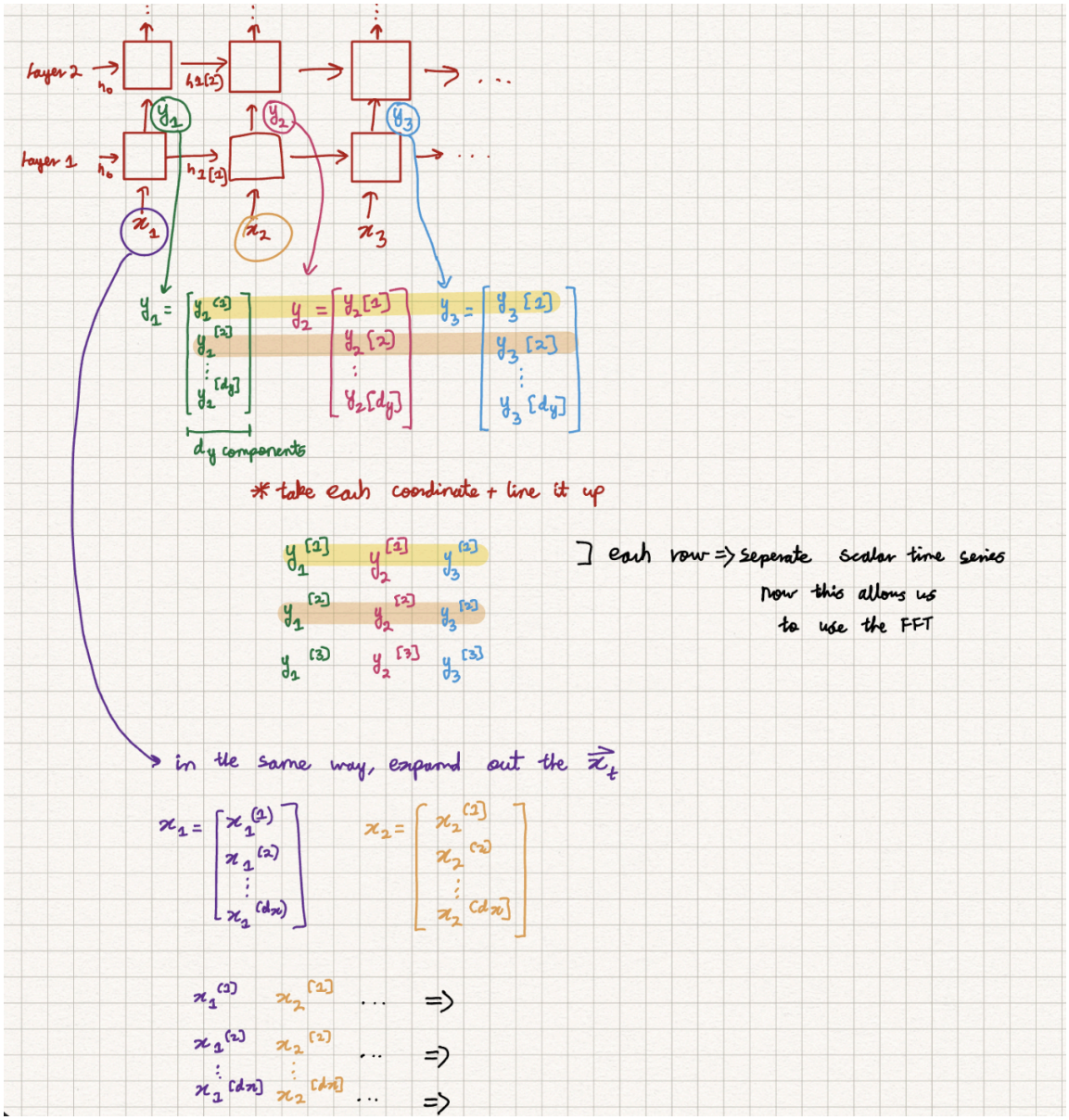
- x_t has dimension d_x → **d_x scalar sequences**
- y_t has dimension d_y → **d_y scalar sequences**
- SSM unrolling makes each output sequence equal to **a sum over d_x convolutions**
- The filters come from CA^kB
- This is why the equation is:

$$\text{Seq}(y) = \sum_{\ell=1}^{d_x} F_\ell * \text{Seq}(x_\ell).$$

If you'd like, I can draw:

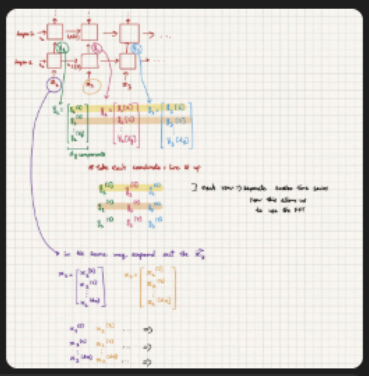
🟢 A full diagram showing state update flow
🔵 How diagonalizing A makes these convolutions independent
🔴 How FFT makes them fast

Just tell me!



This was the diagram I drew so far as I was interacting with ChatGPT

The diagram before helped me understand the overall flow, but I was still unsure why we are summing over d_x instead of the timesteps t



still getting a bit confused. This is the diagram I have right now. so get sequence based on each coordinate of x across time. then you pass it in conv filter? also why is the summation up till dx instead of the t (the number of inputs we got so far)

Great — your diagram is **very close** to the full SSM picture, and your questions pinpoint the **two crucial confusions**:

❓ **(1) Why do we take each coordinate of x across time and form a sequence?**

❓ **(2) Why does the formula sum over d_x instead of summing over time steps t?**

Let me answer both in a **clear, layered explanation** tied directly to your diagram.

✅ FIRST: What your diagram does correctly

You are correctly showing two things:

✔ **Each output vector y_t has d_y coordinates**

→ so lining them up across time gives **d_y scalar sequences.**

✔ **Each input vector x_t has d_x coordinates**

→ lining them up across time gives **d_x scalar sequences.**

This is the correct mental model.

Your diagram (with the rows of y^1, y^2, ...) is EXACTLY the "component-by-component" decomposition that S4 and general SSMs use.

✅ NOW: The key SSM equation (the convolution form)

An SSM unrolls into **convolution**:

$$y_t = \sum_{k=0}^{t} K_k x_{t-k}$$

where:

$$K_k = CA^kB$$

That formula produces:

- **For each output coordinate i (1...d_y),**
- **and each input coordinate ℓ (1...d_x),**
- **there is a scalar convolution filter:**

$$F_{k,\ell}^{(i)} = \left(CA^kB\right)_{i,\ell}$$

So you get:

$$y^{(i)}[t] = \sum_{\ell=1}^{d_x} \sum_{k=0}^{t} F_{k,\ell}^{(i)}\, x^{(\ell)}[t-k].$$

This is the core equation.

⭐ Now let's answer your *specific questions*.

❓ Q1 — "So we get a sequence for each coordinate of x across time. Then we pass it through a convolution filter?"

✔️ **YES — that is *exactly right*.**

Take input coordinate ℓ:

$$x^{(\ell)}[1], x^{(\ell)}[2], x^{(\ell)}[3], \ldots$$

This scalar sequence is fed through a **1-D convolution filter** $F_{\cdot,\ell}^{(i)}$ to produce its contribution to the i-th output sequence.

Visually:

```less
x¹:   x¹[1] → x¹[2] → x¹[3] → …  ---conv--→  contribution to y⁽ⁱ⁾
x²:   x²[1] → x²[2] → x²[3] → …  ---conv--→  contribution to y⁽ⁱ⁾
...
xᵈˣ: xᵈˣ[1] → xᵈˣ[2] → xᵈˣ[3] → …  ---conv--→  contribution to y⁽ⁱ⁾
```

Then you **sum all contributions** from the d_x input channels to get:

$$y^{(i)} = \sum_{\ell=1}^{d_x} F_{\cdot,\ell}^{(i)} * x^{(\ell)}.$$

Each convolution is **scalar** → so FFT works!

This explanation solidified by understanding and answered by question properly

❓ Q2 — "Why is the summation up to d_x instead of the number of time steps?"

This is the BIG IDEA:

⭐ The summation over d_x indexes input *dimensions*,

NOT time steps.

Let's break down the two summations:

🔷 ✓ Summation over k = 0...t

This is the usual convolution over **past time steps**:

$$y_t = \sum_{k=0}^{t} K_k \, x_{t-k}$$

This part **is summing over past times**.

🔷 ✓ Summation over ℓ = 1...d_x

This sums over **input channels / input dimensions**.

Each input vector x_t has d_x coordinates:

$$x_t = \begin{bmatrix} x^{(1)}[t] \\ x^{(2)}[t] \\ \vdots \\ x^{(d_x)}[t] \end{bmatrix}$$

So for each output dimension i, you must combine **all** d_x input streams.

This is why:

$$y^{(i)}[t] = \underbrace{\sum_{k=0}^{t}}_{\text{sum over time}} \underbrace{\sum_{\ell=1}^{d_x}}_{\text{sum over input dims}} F_{k,\ell}^{(i)} \, x^{(\ell)}[t-k].$$

Your confusion is natural because:

- "t" = time
- "d_x" = number of *components/dimensions*, not time steps

🎯 Intuition: Why two sums?

1️⃣ Time convolution

→ For each input dimension, we convolve across past timesteps.

2️⃣ Channel mixing

→ Then we sum all channels to get the final output stream.

This matches standard multi-channel convolution:

```java
Output channel i = sum over input channels (conv(kernel_{i,l}, input_l))
```

Copy code

📌 YOUR DIAGRAM IS ALMOST SHOWING THIS

Let me rewrite it more clearly:

You already did:

1. Each y coordinate becomes a scalar sequence
2. Each x coordinate becomes a scalar sequence

Next step:

For each output coordinate (row):

```
r                                                                    Copy code

y¹ = F₁⁽¹⁾ * x¹   +   F₂⁽¹⁾ * x²   + … +   F_dₓ⁽¹⁾ * x_dₓ
y² = F₁⁽²⁾ * x¹   +   F₂⁽²⁾ * x²   + … +   F_dₓ⁽²⁾ * x_dₓ
...
y_dᵧ = F₁⁽ᵈʸ⁾ * x¹ +   ... +        F_dₓ⁽ᵈʸ⁾ * x_dₓ
```

Each **row** is a scalar convolution over time.
Each **column** corresponds to an input coordinate.
The **sum over columns** corresponds to summation over d_x.

🧨 FINAL SUMMARY

✔️ **You create a scalar sequence for each coordinate of x.**

✔️ **You convolve each with a scalar filter.**

✔️ **You sum over all d_x input channels.**

✔️ **You sum over past time k inside the convolution.**

✔️ **That's why the formula has two summations:**

$$y^{(i)}[t] = \sum_{\ell=1}^{d_x} \sum_{k=0}^{t} F_{k,\ell}^{(i)}\, x^{(\ell)}[t-k].$$

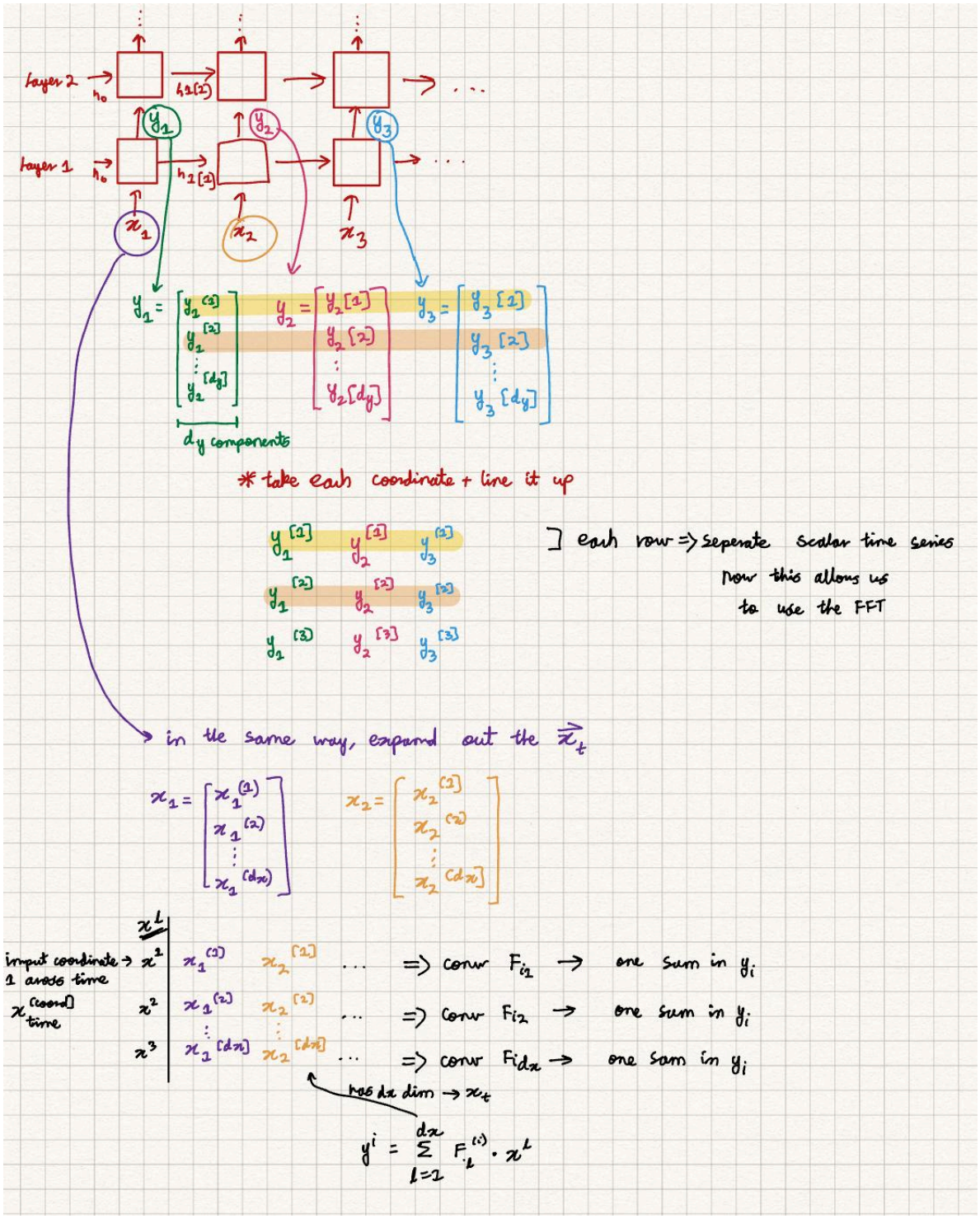
This is 100% consistent with your diagram.

If you want, I can **redraw your diagram** into a clean professional schematic showing:

- x sequences per coordinate
- convolution filters
- channel mixing
- output sequences

Just say "clean schematic please."

This is the final diagram I had:



Overall this interaction definitely helped me better understand the FFT trick used in SSMs and why they are important to reduce the runtime from O(T^2) to O(TlogT)